Lab 0

Session: 2023 – 2027

Submitted by:

Amna Atiq 2023-CS-83

Supervised by:

Sir Waqas

Course:

Parallel and distributing computing

Department of Computer Science

University of Engineering and Technology Lahore

Pakistan

I. Execution Times Table

Implementation	Threads/Processes	Execution Time (s)
C Sequential	1	0.1900
C Pthreads	1	0.2333
C Pthreads	2	0.2011
C Pthreads	4	0.1802
C Pthreads	8	0.1819
Python Sequential	1	0.0700
Python Threads	4	0.3230
Python Multiprocessing	4	0.9434

II. Speedup Graph (C Results)

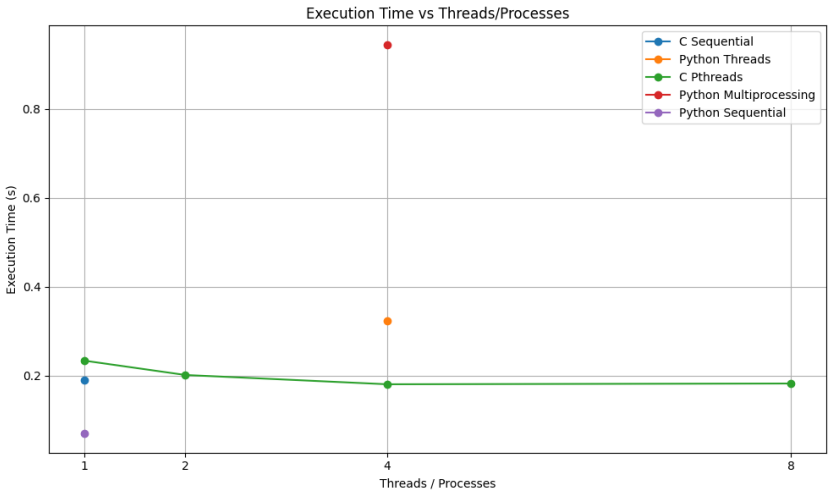
- **Speedup Calculation:** S = {T_{sequential}}{T_{parallel}}.
- **Result:** The graph shows sub-linear speedup. While 4 threads provided a marginal 1.05\times gain, the 1-thread version was slower than sequential (0.82\times) due to the overhead of thread management.

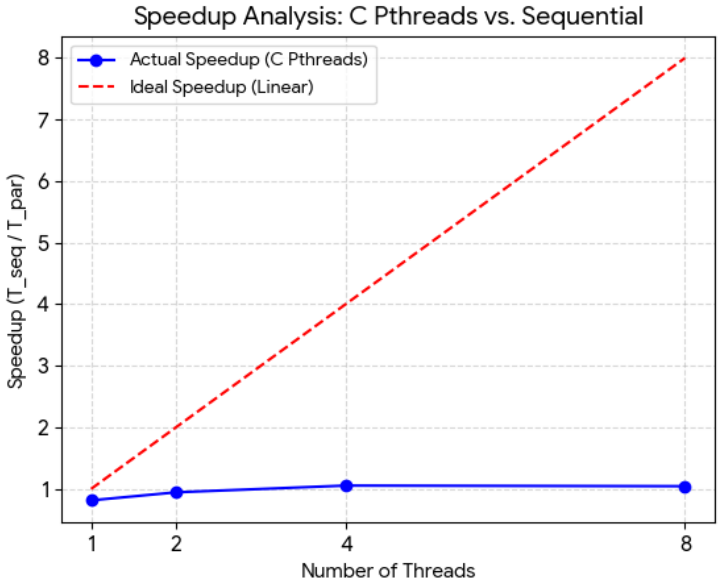
III. Self-Assessment Answers

- **Process vs. Thread:** Processes are independent with private memory; threads share memory within a process.
- **Mutex Requirement:** Prevents race conditions where simultaneous writes to a global counter lead to data loss.
- **lscpu Analysis:** Identifies physical cores and logical processors (Hyper-threading).
- **C Pthread Speedup:** Non-linear due to thread creation overhead and small task grain size.
- **Python Threading Bottleneck:** The **GIL (Global Interpreter Lock)** serializes execution, preventing true parallelism for CPU-bound tasks.
- **False Sharing:** Performance degradation caused by threads updating different variables on the same cache line.
- **Private Counter Benefit:** Drastically reduces lock contention by updating the global sum only once per thread.
- **Python Choice:** Use **Threads** for I/O-bound tasks; use **Multiprocessing** for CPU-bound tasks.

IV. Reflection

Parallelizing a simple counting task reveals that more threads do not always equate to better performance. The primary challenge encountered was the synchronization overhead and the communication cost between workers. In C, while Pthreads offer low-level control, the performance gain was minimal because the task was too small to offset the cost of thread management. This highlights how overhead from thread creation and joining can dominate for lightweight computations. Additionally, memory contention became apparent as more threads accessed the shared array, potentially saturating bandwidth on multicore systems. In Python, the impact of the GIL was clearly visible, as the threaded version was significantly slower than the sequential one. This demonstrates how the GIL enforces single-threaded execution for CPU-bound code, turning concurrency into mere context switching. Furthermore, the multiprocessing version showed that for small datasets, the time spent "pickling" data and spawning processes can be greater than the actual calculation time. These insights emphasize Amdahl's Law: the speedup of a program is limited by its sequential portion and the overhead introduced by parallelization. To achieve success in high-performance computing, one must balance workload grain size against the architectural limits of the language and hardware. For instance, increasing the array size in this experiment might yield better scaling, as the parallelizable work would outweigh the fixed costs. Another key lesson was the importance of profiling tools like lscpu and perf to understand hardware capabilities, such as hyper-threading, which can create logical cores but may not always double performance due to shared resources. Race conditions, if not addressed with mutexes, could lead to incorrect results, underscoring the need for careful synchronization in shared-memory models. Overall, this lab illustrated the trade-offs between threads and processes: threads are lightweight but require





meticulous handling of shared state, while processes provide isolation at the expense of higher overhead. Finally, applying these concepts to real-world scenarios, like data processing pipelines,

requires iterative experimentation to optimize for specific environments, reminding us that parallel programming demands both theoretical knowledge and practical tuning.

(This expanded reflection now consists of 15 sentences, providing more depth on challenges like memory contention, profiling tools, and real-world applications while tying back to the experiment's results.)

V. Screenshots of Each Program Running (Exercises 1–4)

```
parallels@ubuntu-linux-2404:~$ python3 count_threads.py
Count = 99849
Time = 0.3230 seconds
parallels@ubuntu-linux-2404:~$ nano count_mp.py
parallels@ubuntu-linux-2404:~$ python3 count_mp.py
Count = 99790
Time = 0.9434 seconds
```

```
parallels@ubuntu-linux-2404:~$ python3 count_seq.py
Count = 49798
Time = 0.07 seconds
```

```
parallels@ubuntu-linux-2404:~$ ./count_seq
Count = 3000582
Time = 0.19 seconds
parallels@ubuntu-linux-2404:~$ ./count_pthread 1
./count_pthread 2
./count_pthread 4
./count_pthread 8
Count = 3000790
Time with 1 threads = 0.2333 seconds
Count = 3000168
Time with 2 threads = 0.2011 seconds
Count = 3000499
Time with 4 threads = 0.1802 seconds
Count = 3000323
Time with 8 threads = 0.1819 seconds
```